Sri Lanka Institute of Information Technology

IT3012: Intelligent Agents - Lab 3

Y3.S1.WD.AI.01.02

Malabe Campus

Student ID -	IT24103099
Student Name -	Uthpala P.B.H
Module Name -	Intelligent Agents
Module Number -	IT3012
LabSheet Number -	3
LabSheet Date -	2026/August/17

Step 1.1: Exposing the World Model

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.

2. Open visual_grid_game.py .

3. Locate the get_percept(self) method.

4. Add three new keys to the dictionary to pass the global state to the agent: 'grid_size': (self.width, self.height) 'walls': list(self.walls) 'all_food': list(self.food_positions)

```
        # Check whether food is on the current cell
        food_here = tuple(self.agent_pos) in self.food_positi

        return {
            'wall_ahead': wall_ahead,
            'food_here': food_here,
            'grid_size': (self.width, self.height),
            'walls': list(self.walls),
            'all_food': list(self.food_positions)
        }
```

Step 1.2: Implementing BFS, DFS, and UCS

1. Open agent.py and create a new class called SearchAgent .

2. Import required structures: from collections import deque and import heapq .

3. BFS: Create def bfs_search(...) . Use a FIFO queue (deque.popleft()). This explores the shallowest nodes first.

4. DFS: Create def dfs_search(...) . Use a LIFO stack (list.pop()). This explores the deepest nodes first.

5. UCS: Create def ucs_search(...) . Use a Priority Queue (heapq.heappop()) ordered by the total path cost $g(n)$.

6. For all three algorithms, you must maintain a reached set (or visited array) to track explored states. This converts your algorithm from a Tree Search to a Graph Search, preventing infinite loops.

```python
def bfs_search(self, start, goal, walls, grid_size):
    """Breadth-First Search."""

    queue = deque()
    queue.append((start, []))

    reached = {start}

    while queue:
        current, path = queue.popleft()

        # Goal found
        if current == goal:
            return path

        x, y = current

        neighbors = [
            ((x, y + 1), 'Up'),
            ((x, y - 1), 'Down'),
            ((x - 1, y), 'Left'),
            ((x + 1, y), 'Right')
        ]

        for next_pos, action in neighbors:
            nx, ny = next_pos

            # Check grid boundaries, walls, and visited positions
            if (
                0 <= nx < grid_size[0]
                and 0 <= ny < grid_size[1]
                and next_pos not in walls
                and next_pos not in reached
            ):
                reached.add(next_pos)
                queue.append((next_pos, path + [action]))

    # No path found
    return None
```

```python
def dfs_search(self, start, goal, walls, grid_size):
    """Depth-First Search."""

    stack = []
    stack.append((start, []))

    reached = {start}

    while stack:
        current, path = stack.pop()

        # Goal found
        if current == goal:
            return path

        x, y = current

        neighbors = [
            ((x, y + 1), 'Up'),
            ((x, y - 1), 'Down'),
            ((x - 1, y), 'Left'),
            ((x + 1, y), 'Right')
        ]

        for next_pos, action in neighbors:
            nx, ny = next_pos

            # Check grid boundaries, walls, and visited positions
            if (
                0 <= nx < grid_size[0]
                and 0 <= ny < grid_size[1]
                and next_pos not in walls
                and next_pos not in reached
            ):
                reached.add(next_pos)
                stack.append((next_pos, path + [action]))

    # No path found
    return None
```

```python
def ucs_search(self, start, goal, walls, grid_size):
    """Uniform-Cost Search."""

    priority_queue = []
    heapq.heappush(priority_queue, (0, start, []))

    reached = {start: 0}

    while priority_queue:
        cost, current, path = heapq.heappop(priority_queue)

        # Goal found
        if current == goal:
            return path

        x, y = current

        neighbors = [
            ((x, y + 1), 'Up'),
            ((x, y - 1), 'Down'),
            ((x - 1, y), 'Left'),
            ((x + 1, y), 'Right')
        ]

        for next_pos, action in neighbors:
            nx, ny = next_pos

            # Check grid boundaries and walls
            if (
                0 <= nx < grid_size[0]
                and 0 <= ny < grid_size[1]
                and next_pos not in walls
            ):
                # Every movement has a cost of 1
                new_cost = cost + 1

                # Add if this is a new position
                # or if we found a cheaper path
                if (
```

Step 1.3: Executing and Comparing Offline Plans

1. In your SearchAgent.__init__ , add an empty list: self.plan = [] and a config string self.active_algo = 'BFS' .

```python
class SearchAgent:
    """Agent that uses search algorithms to find a path to food."""

    def __init__(self):
        self.plan = []
        self.active_algo = 'BFS'

    def bfs_search(self, start, goal, walls, grid_size):
        """Breadth-First Search."""
```

2. In sense_and_act(self, percept) , check if self.plan is empty.

3. If empty, find the closest food pellet from percept['all_food'] , execute the search method matching self.active_algo , and store the resulting sequence of actions in self.plan .

4. Return the first action from the plan using return self.plan.pop(0) .

5. Observation Task: Change self.active_algo between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

```python
def sense_and_act(self, percept: dict) -> str:
    """Create a complete plan and execute it one action at a time."""

    # Create a new plan only when there is no current plan
    if not self.plan:

        start = tuple(percept['agent_pos'])
        all_food = percept['all_food']
        walls = set(percept['walls'])
        grid_size = percept['grid_size']

        # No food remains
        if not all_food:
            return 'Stay'

        # Find the closest food using Manhattan distance
        closest_food = min(
            all_food,
            key=lambda food:
                abs(food[0] - start[0]) +
                abs(food[1] - start[1])
        )

        goal = tuple(closest_food)

        # Select the active search algorithm
        if self.active_algo == 'BFS':
            self.plan = self.bfs_search(
                start,
                goal,
                walls,
                grid_size
            )

        elif self.active_algo == 'DFS':
            self.plan = self.dfs_search(
                start,
                goal,
                walls,
                grid_size
            )

        elif self.active_algo == 'UCS':
            self.plan = self.ucs_search(
```

Part 2: Theoretical Evaluation

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

- The State Space is all the possible positions or states that the agent can be in.
- The Search Tree is the paths that the search algorithm explores from the starting position to find the goal.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

- The reached set remembers the positions that the algorithm has already visited. It stops the algorithm from visiting the same position again and again. Without it, DFS could get stuck in a **loop**, such as moving Right → Left → Right → Left repeatedly. It might never reach the food.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

- In our grid, every movement has the same cost of **1**.BFS checks the closest positions first, so when it finds the food, it has found the shortest path.DFS goes as deep as possible along one path before trying another path. Because of this, it can take unnecessary turns and produce a longer path.

4. (Evaluate) If we scaled `visual_grid_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

- BFS uses a lot of memory because it stores many positions in its queue while searching.DFS uses less memory because it mainly follows one path at a time.So, on a very large grid, BFS is more likely to run out of memory. DFS uses less memory, but it may take much longer to find the food because it can explore a bad path first.